

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

```
from nltk.tokenize import word_tokenize
```

```
sample_text = "This is a text ready to tokenize"
token = word_tokenize(sample_text)
print(sample_text)
print(token)
```

```
This is a text ready to tokenize
['This', 'is', 'a', 'text', 'ready', 'to', 'tokenize']
```

```
from nltk.tokenize import TweetTokenizer
sample_text = "This is a tweet @jack #NLP"
tweet_tokenizer = TweetTokenizer()
token = tweet_tokenizer.tokenize(sample_text)
print(sample_text)
print(token)
```

```
This is a tweet @jack #NLP
['This', 'is', 'a', 'tweet', '@jack', '#NLP']
```

```
from nltk.tokenize import sent_tokenize
```

```
sample_text = "This is a sentence. This is another one!\nAnd This is the last sentence."
sentences = sent_tokenize(sample_text)
print(sample_text)
print(sentences)
```

```
This is a sentence. This is another one!
And This is the last sentence.
['This is a sentence.', 'This is another one!', 'And This is the last sentence.']
```

```
from nltk.corpus import stopwords
nltk.download("stopwords")
stopwords_ = set(stopwords.words("english"))
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

```
sample_text = "this is sample text"
tokens = sample_text.split()
clean_tokens = [t for t in tokens if not t in stopwords_]
clean_text = " ".join(clean_tokens)
print(clean_tokens)
print(clean_text)
```

```
['sample', 'text']
sample text
```

✓ Porter Stem

```
from nltk.stem import PorterStemmer
stemmer = PorterStemmer()
```

```
words = ["running", "flies", "happily", "better", "jumps", "happiest"]
stemmed_words = [stemmer.stem(word) for word in words]
print(stemmed_words)
```

```
['run', 'fli', 'happili', 'better', 'jump', 'happiest']
```

✓ Lancaster Stem

```
from nltk.stem import LancasterStemmer
stemmer = LancasterStemmer()
```

```
words = ["running", "flies", "happily", "better", "jumps", "happiest"]
stemmed_words = [stemmer.stem(word) for word in words]
print(stemmed_words)
```

```
['run', 'fli', 'happy', 'bet', 'jump', 'happiest']
```

✓ Snowball Stemmer

```
from nltk.stem import SnowballStemmer
stemmer = SnowballStemmer("english")
```

```
words = ["running", "flies", "happily", "better", "jumps", "happiest"]
stemmed_words = [stemmer.stem(word) for word in words]
print(stemmed_words)
```

```
['run', 'fli', 'happili', 'better', 'jump', 'happiest']
```

✓ RegExp Stemmer

```
from nltk.stem import RegexpStemmer
stemmer = RegexpStemmer("ing$|es$|ly$|er$", min=4)
```

```
stemmed_words = [stemmer.stem(word) for word in words]
print(stemmed_words)
```

```
['runn', 'fli', 'happi', 'bett', 'jumps', 'happiest']
```

✓ WordNet Lemmatizer

```
nltk.download('wordnet')
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
```

```
lemmatized_words = [lemmatizer.lemmatize(word) for word in words]
print(lemmatized_words)
```

```
['running', 'fly', 'happily', 'better', 'jump', 'happiest']
```

✓ TextBlob Lemmatizer

```
pip install textblob
```

```
Requirement already satisfied: textblob in /usr/local/lib/python3.12/dist-packages (0.19.0)
Requirement already satisfied: nltk>=3.9 in /usr/local/lib/python3.12/dist-packages (from textblob) (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk>=3.9->textblob) (8.3.3)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk>=3.9->textblob) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk>=3.9->textblob) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk>=3.9->textblob) (4.67.3)
```

```
from textblob import Word
```

```
lemmatized_words = [Word(word).lemmatize() for word in words]
print(lemmatized_words)
```

```
['running', 'fly', 'happily', 'better', 'jump', 'happiest']
```

✓ Part of Speech Tagging

```
nltk.download("tagsets_json")
```

```
[nltk_data] Downloading package tagsets_json to /root/nltk_data...
[nltk_data] Unzipping help/tagsets_json.zip.
True
```

```
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
```

```
[nltk_data] Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
True
```

```
sentence = "The quick brown fox jump over the lazy dog"
words = nltk.word_tokenize(sentence)
pos_tags = nltk.pos_tag(words)
print("")
print("POS Tags\n", pos_tags, "\n")
```

```
for word, tag in pos_tags:
    print(nltk.help.upenn_tagset(tagpattern=tag))
```

```
POS Tags
[('The', 'DT'), ('quick', 'JJ'), ('brown', 'NN'), ('fox', 'JJ'), ('jump', 'NN'), ('over', 'IN'), ('the', 'DT'), ('lazy', 'JJ'), ('dog', 'NN')]

DT: determiner
all an another any both del each either every half la many much nary
neither no some such that the them these this those
None
JJ: adjective or numeral, ordinal
third ill-mannered pre-war regrettable oiled calamitous first separable
ectoplasmic battery-powered participatory fourth still-to-be-named
multilingual multi-disciplinary ...
None
NN: noun, common, singular or mass
common-carrier cabbage knuckle-duster Casino afghan shed thermostat
investment slide humour falloff slick wind hyena override subhumanity
machinist ...
None
JJ: adjective or numeral, ordinal
third ill-mannered pre-war regrettable oiled calamitous first separable
ectoplasmic battery-powered participatory fourth still-to-be-named
multilingual multi-disciplinary ...
None
NN: noun, common, singular or mass
common-carrier cabbage knuckle-duster Casino afghan shed thermostat
investment slide humour falloff slick wind hyena override subhumanity
machinist ...
None
IN: preposition or conjunction, subordinating
astride among upon whether out inside pro despite on by throughout
below within for towards near behind atop around if like until below
next into if beside ...
None
DT: determiner
all an another any both del each either every half la many much nary
neither no some such that the them these this those
None
JJ: adjective or numeral, ordinal
third ill-mannered pre-war regrettable oiled calamitous first separable
ectoplasmic battery-powered participatory fourth still-to-be-named
multilingual multi-disciplinary ...
None
NN: noun, common, singular or mass
common-carrier cabbage knuckle-duster Casino afghan shed thermostat
investment slide humour falloff slick wind hyena override subhumanity
machinist ...
None
```

```
pip install svgling
```

```
Collecting svgling
  Downloading svgling-0.5.0-py3-none-any.whl.metadata (7.4 kB)
Collecting svgwrite (from svgling)
  Downloading svgwrite-1.4.3-py3-none-any.whl.metadata (8.8 kB)
Downloading svgling-0.5.0-py3-none-any.whl (31 kB)
Downloading svgwrite-1.4.3-py3-none-any.whl (67 kB)
----- 67.1/67.1 kB 3.9 MB/s eta 0:00:00
Installing collected packages: svgwrite, svgling
Successfully installed svgling-0.5.0 svgwrite-1.4.3
```

```
nltk.download("maxent_ne_chunker")
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
```

```
[nltk_data] Downloading package maxent_ne_chunker to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping chunkers/maxent_ne_chunker.zip.
[nltk_data] Downloading package maxent_ne_chunker_tab to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping chunkers/maxent_ne_chunker_tab.zip.
[nltk_data] Downloading package words to /root/nltk_data...
[nltk_data] Unzipping corpora/words.zip.
True
```

```
from nltk.tag import Tagger
```

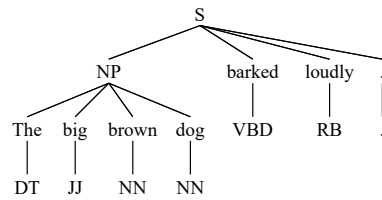
from nltk.tree import Tree

```
sentence = "The big brown dog barked loudly."
words = nltk.word_tokenize(sentence)
pos_tags = nltk.pos_tag(words)

chunked = nltk.ne_chunk(pos_tags)

reg_exp = "NP: {<DT>?<JJ>*<NN>*}"
rp = nltk.RegexpParser(reg_exp)
result = rp.parse(chunked)

result
```



Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.